

“The Game Beneath the Game: The Evolution of Cheats and Anti-Cheat Systems in Counter-Strike 2”

Author: Pablo Valiente

Supervisor: Dr. Antonio Nappa

Universidad Carlos III de Madrid

Academic Year: 2024–2025

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Table of Contents

1. Motivation and Context
2. Cheating: History, Ecosystem, and Ethics
3. Anti-Cheat Technologies and Milestones
4. Case Study: Counter-Strike 2
5. Technical Analysis: Injection and Evasion
6. Game Logic Manipulation and Visual Features
7. Final Reflections and Future Directions

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Motivation

My interest in this topic began around six years ago, driven by a passion for competitive video games.

- **Curiosity:** Encountering cheaters in ranked matches often led me to wonder: *"How did they do it?"*
- **Exploration:** This question gradually introduced me to reverse engineering, memory analysis, and anti-cheat evasion.
- **Background:** Over the years, I acquired technical knowledge through communities, experiments, and academic training.
- **Objective:** To merge this journey with a rigorous academic study by analyzing and implementing real cheats to test detection systems.

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic triggerbots to advanced kernel-level exploits.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic triggerbots to advanced kernel-level exploits.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic triggerbots to advanced kernel-level exploits.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic triggerbots to advanced kernel-level exploits.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Understanding Cheating in Online Games

Cheating undermines fairness and security in competitive online games.

- **Definition:** Any software or hardware manipulation that grants unfair advantage.
- **Anti-Cheat:** Monitors and enforces integrity at runtime, acting like an **EDR** by detecting anomalous behavior, unauthorized memory access, or code injection in protected processes.
- **Spectrum:** From basic triggerbots to advanced kernel-level exploits.
- **Impact:** Threatens matchmaking integrity and industry revenue.
- **Context:** Now part of the broader cybersecurity landscape.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

Origins of Cheating: From Debugging to Subversion

Cheating emerged from tools meant for testing and evolved into manipulation.

- **Debug Codes:** Became public cheat codes (e.g., Konami Code).
- **GameShark:** Modified RAM directly on console hardware.
- **Cheat Engine:** Enabled runtime memory editing on PC.
- **x64dbg and ReClass.NET:** Allowed advanced memory mapping, pointer tracing, and class reconstruction.
- **Advanced Tooling:** Debuggers like x64dbg, memory editors, and tools such as ReClass.NET enabled deep inspection of structures and dynamic memory regions.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort—fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort—fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort—fueling a sustainable market.

From Hacks to Services: The Turning Point

The early 2010s marked a major shift in how cheats were created, distributed, and perceived.

- **Community Forums:** Platforms like UnKnoWnCheaTs fostered knowledge sharing and tutorials.
- **Toolchains:** Public tools and open-source loaders lowered the barrier to entry.
- **Subscription Models:** Private cheats became commercialized with regular updates and support.
- **User Demand:** Players sought not just to win, but to dominate with minimal effort—fueling a sustainable market.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups—offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by technical forums, Discord channels, and reverse engineering collectives.
- **Ethical Ambiguity:** Some actors claim educational or research motives—blurring lines between analysis and abuse.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups—offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by technical forums, Discord channels, and reverse engineering collectives.
- **Ethical Ambiguity:** Some actors claim educational or research motives—blurring lines between analysis and abuse.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups—offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by technical forums, Discord channels, and reverse engineering collectives.
- **Ethical Ambiguity:** Some actors claim educational or research motives—blurring lines between analysis and abuse.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups—offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by technical forums, Discord channels, and reverse engineering collectives.
- **Ethical Ambiguity:** Some actors claim educational or research motives—blurring lines between analysis and abuse.

Cheating Today: A Professionalized Ecosystem

Cheating has evolved from hobbyist tools to a mature ecosystem that mirrors legitimate software development.

- **Commercialization:** Cheat providers operate like startups—offering licensing, tiered features, and customer support.
- **Stealth Technology:** Use of polymorphism, virtualization, and driver-level code to evade detection.
- **Infrastructure:** Loaders, obfuscators, HWID spoofers, and encrypted update channels ensure persistence.
- **Community:** Development is sustained by technical forums, Discord channels, and reverse engineering collectives.
- **Ethical Ambiguity:** Some actors claim educational or research motives—blurring lines between analysis and abuse.

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

Anti-Cheat Systems Timeline

Anti-cheat systems have evolved from basic client-side tools to complex, kernel-level detectors. Each marked a significant shift in visibility, access, or enforcement.

System	Year	Mode	Milestone or Innovation
PunkBuster	2000	User	First real-time memory scan and screenshot system
VAC	2002	User	Delayed ban model to obfuscate detection methods
BattleEye	2004	Kernel	Early adoption of kernel-mode drivers
FairFight	2012	Server	Behavioral/statistical detection at server level
EAC	2018	Kernel	Real-time scanning with driver-level protection
Vanguard	2020	Kernel	Persistent always-on kernel driver (controversial)
VAC Live	2023	Hybrid	Real-time detection with low-latency enforcement

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why Counter-Strike 2?

Counter-Strike 2 represents a prime environment to study modern cheating.

- **Large Player Base:** Over 1 million monthly active users in 2024–2025.
- **High Competitive Stakes:** Ranked ladders, esports scene, and skin economy intensify incentives.
- **Persistent Cheating Problem:** VAC bans, report abuse, and public complaints highlight widespread issues.
- **Technical Maturity:** CS2 is built on Source 2 with new anti-cheat features and detection paradigms.
- **Public Data:** Rich dataset from platforms like SteamCharts and CSStats enables evidence-based analysis.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them—even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them—even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them—even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

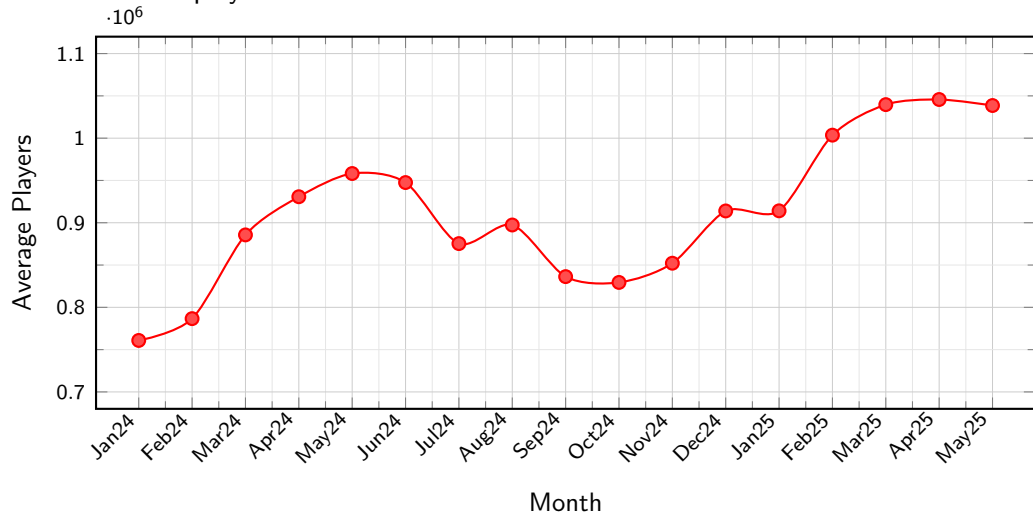
Why FPS Games Are Especially Vulnerable

First-person shooters present unique architectural weaknesses that make them highly susceptible to game manipulation.

- **Client-Side Rendering:** The game client must access enemy positions to render them—even through walls.
- **Memory Residency:** Positional and player state data exist in memory and can be externally read or modified.
- **Contrast With Strategy Games:** RTS or turn-based titles store game logic server-side, making manipulation far more difficult.
- **Exploitation Vector:** These FPS traits enable features like ESP, aimbots, and recoil control by hijacking user input or memory data.

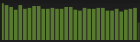
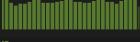
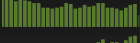
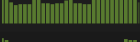
CS2 Player Base Growth (2024–2025)

To contextualize the scale of CS2's ecosystem, the following graph illustrates the evolution of its player base based on SteamCharts data.



Steam Top 10: July 7, 2025

Top Games By Current Players

	Name	Current Players	Last 30 Days	Peak Players	Hours Played
1.	Counter-Strike 2	1,222,782		1,725,815	729,749,131
2.	PUBG: BATTLEGROUNDS	581,616		778,327	226,931,487
3.	Dota 2	508,454		648,095	300,471,647
4.	Bongo Cat	169,028		192,501	110,247,318
5.	Delta Force	164,154		177,775	60,062,183
6.	Apex Legends	150,271		250,796	73,537,334
7.	Stardew Valley	133,590		156,589	47,739,319
8.	Rust	129,532		187,345	73,879,862
9.	Wallpaper Engine	120,513		138,702	61,757,771
10.	Grand Theft Auto V Legacy	90,856		112,862	53,501,676

[More](#)

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system—VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system—VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system—VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

VAC: Valve's Anti-Cheat Strategy

Since the early days of Counter-Strike, Valve has maintained its own anti-cheat system—VAC (Valve Anti-Cheat).

- **Consistency:** VAC has been used since CS 1.6 and evolved alongside the series.
- **User Mode Limitation:** VAC operates entirely in user space, unlike newer kernel-mode anticheats.
- **Detection Philosophy:** Prefers silent detection with delayed banwaves to avoid tipping off cheat developers.
- **Upcoming Evolution:** Valve announced integration of **machine learning models** into VAC Live to detect behavioral patterns and cheating heuristics in real time.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players in CS:GO and, later, CS2. Its historical and functional design aims to reduce disruption, but also introduces new detection and exploitation challenges.

- **Introduction:** Trust Factor was introduced in late 2017 to improve matchmaking integrity in CS:GO and later adopted in CS2.
- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups cheaters with cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players in CS:GO and, later, CS2. Its historical and functional design aims to reduce disruption, but also introduces new detection and exploitation challenges.

- **Introduction:** Trust Factor was introduced in late 2017 to improve matchmaking integrity in CS:GO and later adopted in CS2.
- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups cheaters with cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players in CS:GO and, later, CS2. Its historical and functional design aims to reduce disruption, but also introduces new detection and exploitation challenges.

- **Introduction:** Trust Factor was introduced in late 2017 to improve matchmaking integrity in CS:GO and later adopted in CS2.
- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups cheaters with cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Trust Factor and Detection Challenges

Valve's Trust Factor system, introduced in late 2017, fundamentally changed matchmaking by leveraging a hidden reputation metric to segregate players in CS:GO and, later, CS2. Its historical and functional design aims to reduce disruption, but also introduces new detection and exploitation challenges.

- **Introduction:** Trust Factor was introduced in late 2017 to improve matchmaking integrity in CS:GO and later adopted in CS2.
- **Function:** Uses behavioral data, report history, account characteristics, and system metrics to infer player reliability.
- **Segregation Strategy:** Groups cheaters with cheaters, and legitimate players with each other, to reduce disruptive overlap.
- **Opacity:** The exact calculation is hidden, preventing transparency or dispute of unfair evaluations.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**.
- **False Positives: Unjustified bans** occur, often with no appeals process or recourse.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**.
- **False Positives: Unjustified bans** occur, often with no appeals process or recourse.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**.
- **False Positives: Unjustified bans** occur, often with no appeals process or recourse.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

Economic and Psychological Impact

Cheating in CS2 generates profound economic losses and undermines player well-being.

- **Skin Markets: VAC bans** can instantly invalidate cosmetic items worth **thousands of euros**.
- **False Positives: Unjustified bans** occur, often with no appeals process or recourse.
- **Morale: Rampant cheating** erodes competitive spirit, trust, and perceived fairness.
- **Retention: Frustrated players** abandon the game, shrinking and destabilizing the community.

CS2 Player Base vs. Cheating Incidents

The true scale of cheating is revealed by player statistics and detection data.

- **Player Base: Over 1 million** monthly active users in 2024–2025.
- **VAC Reports: 32,000+ matches** with banned players detected in early 2024.
- **Low Profile: 97% of matches** featured only one silent cheater.
- **Stealth: Modern cheats** prioritize **invisibility** and subtlety over blatant advantage.

CS2 Player Base vs. Cheating Incidents

The true scale of cheating is revealed by player statistics and detection data.

- **Player Base:** Over 1 million monthly active users in 2024–2025.
- **VAC Reports:** 32,000+ matches with banned players detected in early 2024.
- **Low Profile:** 97% of matches featured only one silent cheater.
- **Stealth:** Modern cheats prioritize **invisibility** and subtlety over blatant advantage.

CS2 Player Base vs. Cheating Incidents

The true scale of cheating is revealed by player statistics and detection data.

- **Player Base:** Over 1 million monthly active users in 2024–2025.
- **VAC Reports:** 32,000+ matches with banned players detected in early 2024.
- **Low Profile:** 97% of matches featured only one silent cheater.
- **Stealth:** Modern cheats prioritize invisibility and subtlety over blatant advantage.

CS2 Player Base vs. Cheating Incidents

The true scale of cheating is revealed by player statistics and detection data.

- **Player Base:** Over 1 million monthly active users in 2024–2025.
- **VAC Reports:** 32,000+ matches with banned players detected in early 2024.
- **Low Profile:** 97% of matches featured only one silent cheater.
- **Stealth:** Modern cheats prioritize **invisibility** and subtlety over blatant advantage.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player—often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player—often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player—often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Quantifying the Impact: Cheating and Player Statistics

Beyond anecdotal evidence, empirical data helps us understand the scope of cheating in CS2.

- **Player Growth:** Monthly average players increased from 760k to over 1 million (SteamCharts, Jan 2024 – May 2025).
- **Delayed Enforcement:** 28% of VAC bans occur within 24h of last match; average delay is 20 days.
- **Cheating Embedded:** 97% of affected matches had only one banned player—often undetected in real time.
- **No Case Farming Evidence:** Cheaters are not primarily motivated by item drops but by gameplay disruption or rank inflation.

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

DLL Injection and Manual Mapping

- DLLs are injected using classic methods (LoadLibrary, CreateRemoteThread).
- Manual Mapping avoids detection: no LoadLibrary calls, no module list entry.
- Allows stealthy loading of cheat code into memory.

DLL Injection and Manual Mapping

- DLLs are injected using classic methods (LoadLibrary, CreateRemoteThread).
- Manual Mapping avoids detection: no LoadLibrary calls, no module list entry.
- Allows stealthy loading of cheat code into memory.

DLL Injection and Manual Mapping

- DLLs are injected using classic methods (LoadLibrary, CreateRemoteThread).
- Manual Mapping avoids detection: no LoadLibrary calls, no module list entry.
- Allows stealthy loading of cheat code into memory.

Manual Mapping Overview

Key Steps:

1. Allocate memory in target process.
2. Map sections of the DLL manually.
3. Resolve imports and apply relocations.
4. Execute DLL entry point stealthily.

Manual Mapping Overview

Key Steps:

1. Allocate memory in target process.
2. Map sections of the DLL manually.
3. Resolve imports and apply relocations.
4. Execute DLL entry point stealthily.

Manual Mapping Overview

Key Steps:

1. Allocate memory in target process.
2. Map sections of the DLL manually.
3. Resolve imports and apply relocations.
4. Execute DLL entry point stealthily.

Manual Mapping Overview

Key Steps:

1. Allocate memory in target process.
2. Map sections of the DLL manually.
3. Resolve imports and apply relocations.
4. Execute DLL entry point stealthily.

VAC Detection and Countermeasures

- VAC detects signatures, thread start addresses, and VMT hooks.
- Developers evade detection with string encryption, polymorphism, and fileless techniques.
- It's a constant game of adaptation and evasion.

VAC Detection and Countermeasures

- VAC detects signatures, thread start addresses, and VMT hooks.
- Developers evade detection with string encryption, polymorphism, and fileless techniques.
- It's a constant game of adaptation and evasion.

VAC Detection and Countermeasures

- VAC detects signatures, thread start addresses, and VMT hooks.
- Developers evade detection with string encryption, polymorphism, and fileless techniques.
- It's a constant game of adaptation and evasion.

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

Function Hooking and MinHook

- Hooks intercept engine calls (e.g., render, input).
- MinHook provides safe trampoline hooking.
- Allows modification without altering engine behavior.

Function Hooking and MinHook

- Hooks intercept engine calls (e.g., render, input).
- MinHook provides safe trampoline hooking.
- Allows modification without altering engine behavior.

Function Hooking and MinHook

- Hooks intercept engine calls (e.g., render, input).
- MinHook provides safe trampoline hooking.
- Allows modification without altering engine behavior.

Entity List and Memory Traversal

- Entities are managed via paged handles.
- We hook OnAddEntity/OnRemoveEntity to track game state.
- Static and dynamic analysis reveal entity structure.

Entity List and Memory Traversal

- Entities are managed via paged handles.
- We hook OnAddEntity/OnRemoveEntity to track game state.
- Static and dynamic analysis reveal entity structure.

Entity List and Memory Traversal

- Entities are managed via paged handles.
- We hook OnAddEntity/OnRemoveEntity to track game state.
- Static and dynamic analysis reveal entity structure.

Chams and ESP

- Chams: custom materials injected to render models through walls.
- ESP: overlays entity data (e.g., health, name, position).
- Fully internal: no overlays or external rendering.

Chams and ESP

- Chams: custom materials injected to render models through walls.
- ESP: overlays entity data (e.g., health, name, position).
- Fully internal: no overlays or external rendering.

Chams and ESP

- Chams: custom materials injected to render models through walls.
- ESP: overlays entity data (e.g., health, name, position).
- Fully internal: no overlays or external rendering.

Visual Examples: Chams Materials

- Left: Bloom-based Glow Chams
- Right: Fresnel edge-highlight Chams

Visual Examples: Chams Materials

- Left: Bloom-based Glow Chams
- Right: Fresnel edge-highlight Chams

Aimbot and Anti-Aim

- Aimbot: computes angle vectors and modifies view orientation.
- Anti-Aim: misleads enemies and aimbots by desyncing player model.
- Both features are configurable and stealthy.

Aimbot and Anti-Aim

- Aimbot: computes angle vectors and modifies view orientation.
- Anti-Aim: misleads enemies and aimbots by desyncing player model.
- Both features are configurable and stealthy.

Aimbot and Anti-Aim

- Aimbot: computes angle vectors and modifies view orientation.
- Anti-Aim: misleads enemies and aimbots by desyncing player model.
- Both features are configurable and stealthy.

Outline

Motivation and Context

Cheating: History, Ecosystem, and Ethics

Anti-Cheat Technologies and Milestones

Case Study: Counter-Strike 2

Technical Analysis: Injection and Evasion

Game Logic Manipulation and Visual Features

Final Reflections and Future Directions

Conclusions

- Cheating is both a technical and cultural phenomenon.
- Current anti-cheat approaches are limited by privacy and evasion tactics.
- Dual analysis (offensive + defensive) provides better insight into vulnerabilities.

Conclusions

- Cheating is both a technical and cultural phenomenon.
- Current anti-cheat approaches are limited by privacy and evasion tactics.
- Dual analysis (offensive + defensive) provides better insight into vulnerabilities.

Conclusions

- Cheating is both a technical and cultural phenomenon.
- Current anti-cheat approaches are limited by privacy and evasion tactics.
- Dual analysis (offensive + defensive) provides better insight into vulnerabilities.

- Dynamic schema parsing to replace hardcoded offsets.
- Secure string obfuscation to evade static analysis.
- Generalize to other Source 2 games and integrate with anti-cheat benchmarking.

Future Work

- Dynamic schema parsing to replace hardcoded offsets.
- Secure string obfuscation to evade static analysis.
- Generalize to other Source 2 games and integrate with anti-cheat benchmarking.

- Dynamic schema parsing to replace hardcoded offsets.
- Secure string obfuscation to evade static analysis.
- Generalize to other Source 2 games and integrate with anti-cheat benchmarking.

Thank you for your attention!

Any questions?